Maxflow

Naveen Kumar

Assistant Professor
SVNIT, Surat

October 15, 2026

Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

Motivation

- A directed graph can be interpreted as a **flow network**.
- This helps us solve many **material flow–related problems**.
- **Example:**
  - Material flows from a **source** (producer) to a **sink** (consumer).
  - The source produces material at a steady rate; the sink consumes it at the same rate.
  - Flow networks can model:

- Flow networks can model:
  - Liquids through pipes,
  - Parts through assembly lines,
  - Electric current in circuits, and
  - Information in communication networks.

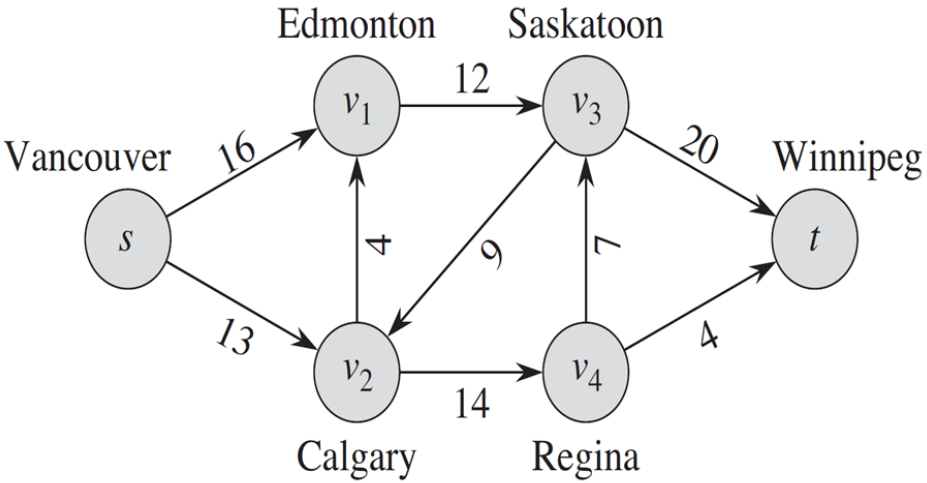


Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

Flow Networks

- Let $G = (V, E)$ be a flow network with a capacity function $c(u, v) \geq 0$. Let s be the source of the network and t be the sink.
- A **flow** in G is a real-valued function

$$f : V \times V \to \mathbb{R}$$

where V is the set of vertices of G and $\mathbb{R}$ is the set of real numbers.

It satisfies the following two properties:

- **1. Capacity Constraint:** For all $u, v \in V$, we require

$$0 \leq f(u, v) \leq c(u, v)$$

i.e., the flow on an edge should not exceed its capacity.

- **2. Flow Conservation:** For all $u \in V - \{s, t\}$, we require

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

i.e., total inflow equals total outflow at every intermediate vertex.

When $(u, v) \notin E$, there is no flow from u to v and hence $f(u, v) = 0$.

Flow Value

- We call the non-negative quantity $f(u, v)$ the **flow** from vertex u to vertex v.
- The **value of the flow** in the network G is defined as:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$$

- Here, s is the **source node**, i.e., total flow **out from the source** minus total flow **into the source**.
- One may ask why we calculate $|f|$ in this way when normally $\sum_{v \in V} f(v, s) = 0$.

$$\sum_{v \in V} f(v, s) = 0$$

- **Note:** When we introduce **residual networks** later, the flow into the source will become significant.

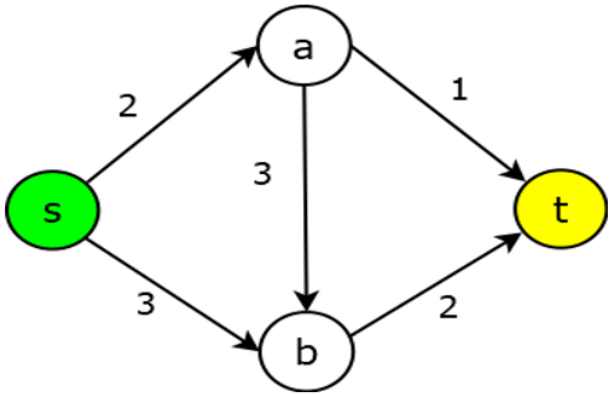
Let's See a Greedy Algorithm

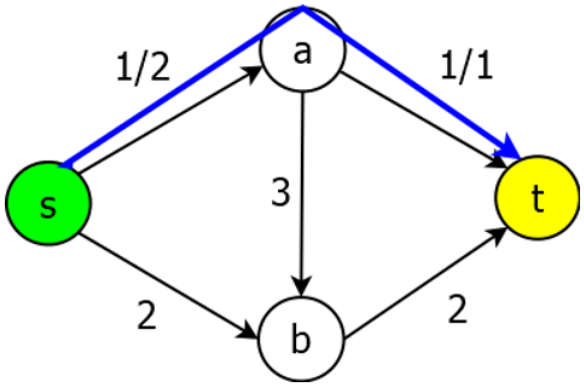
- The Ford–Fulkerson method can be viewed as a **greedy approach** to finding the maximum flow.
- The basic idea is to repeatedly push as much flow as possible along any available s–t path.

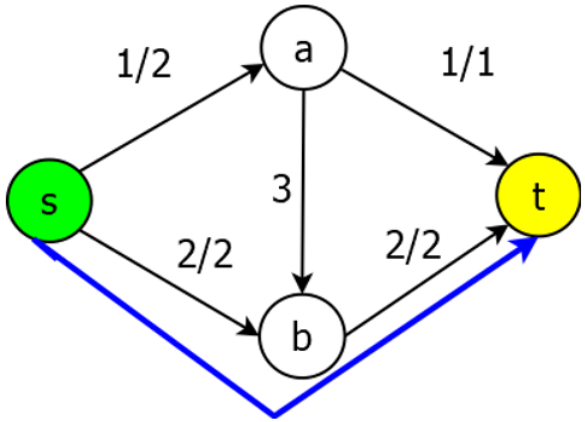
1. Find a path from s to t for which flow exists.
2. Find the flow using the **bottleneck edge** and add this value to the total flow.
3. Update the graph (or residual network) to reflect the new flow values.
4. Repeat Steps 1–3 until no more flow is possible.

This iterative process eventually leads to the maximum flow in the network.

Example



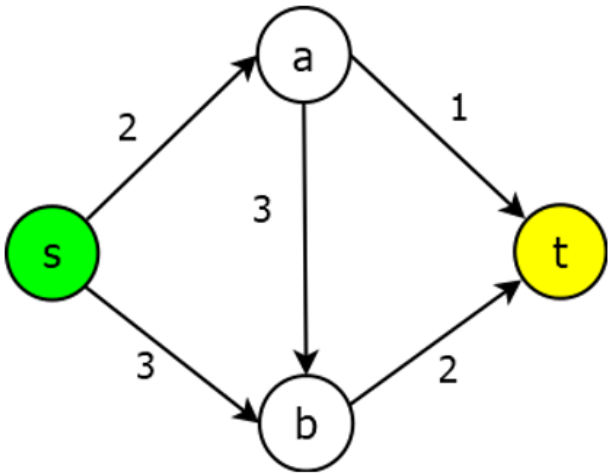


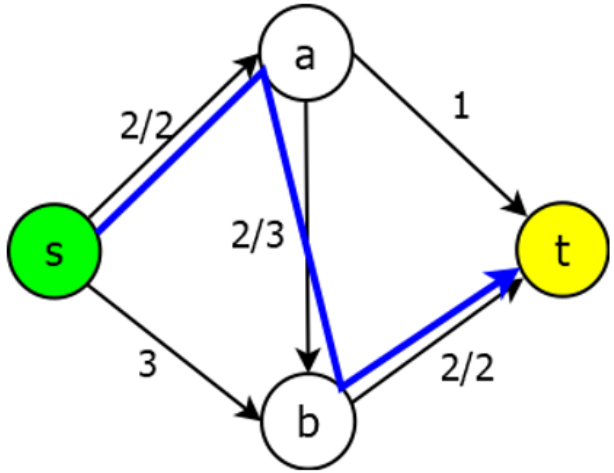


Path-1: s->a->t
Path-2: s->b->t

Total Flow = 3 unit

Example



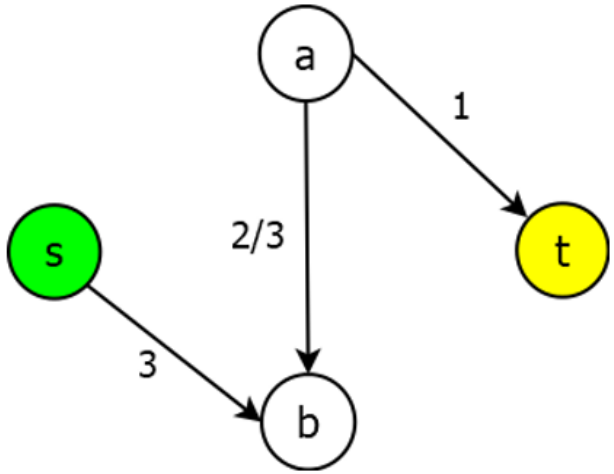


Path-1: s->a->b->t

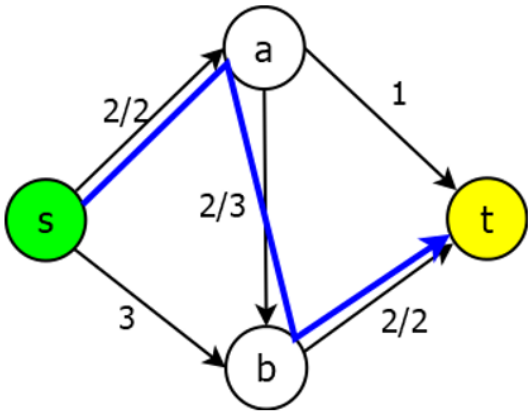
Path-2: Not possible

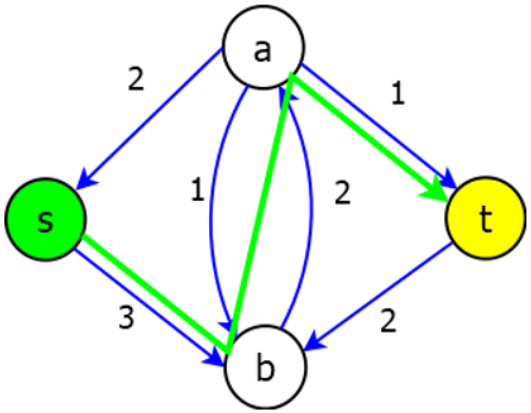
Total Flow = 2 unit

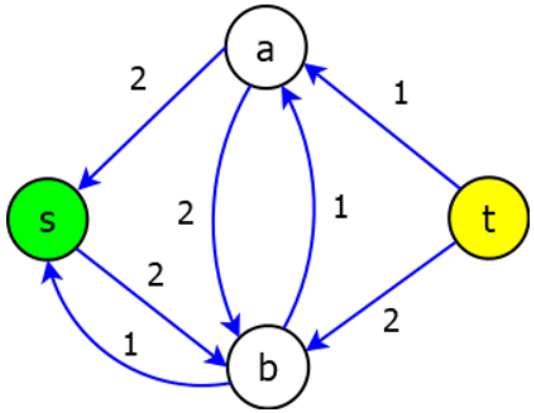
What is the problem???



Example







Total Flow-3

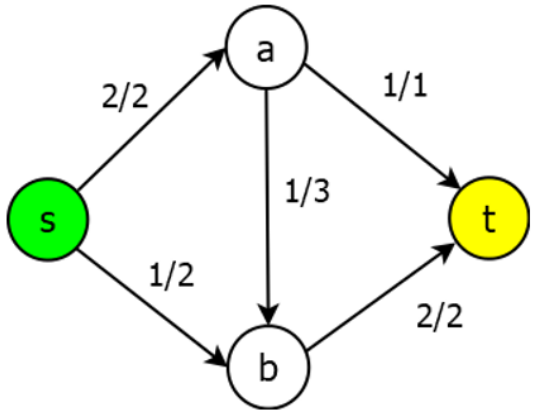


Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

Residual Networks

- Suppose that we have a flow network $G = (V, E)$ with source s and sink t. Let f be a flow in G, and consider a pair of vertices $u, v \in V$.
- We define the *residual capacity* $c_f(u, v)$ by

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v), & \text{if } (u, v) \in E, \\ f(v, u), & \text{if } (v, u) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

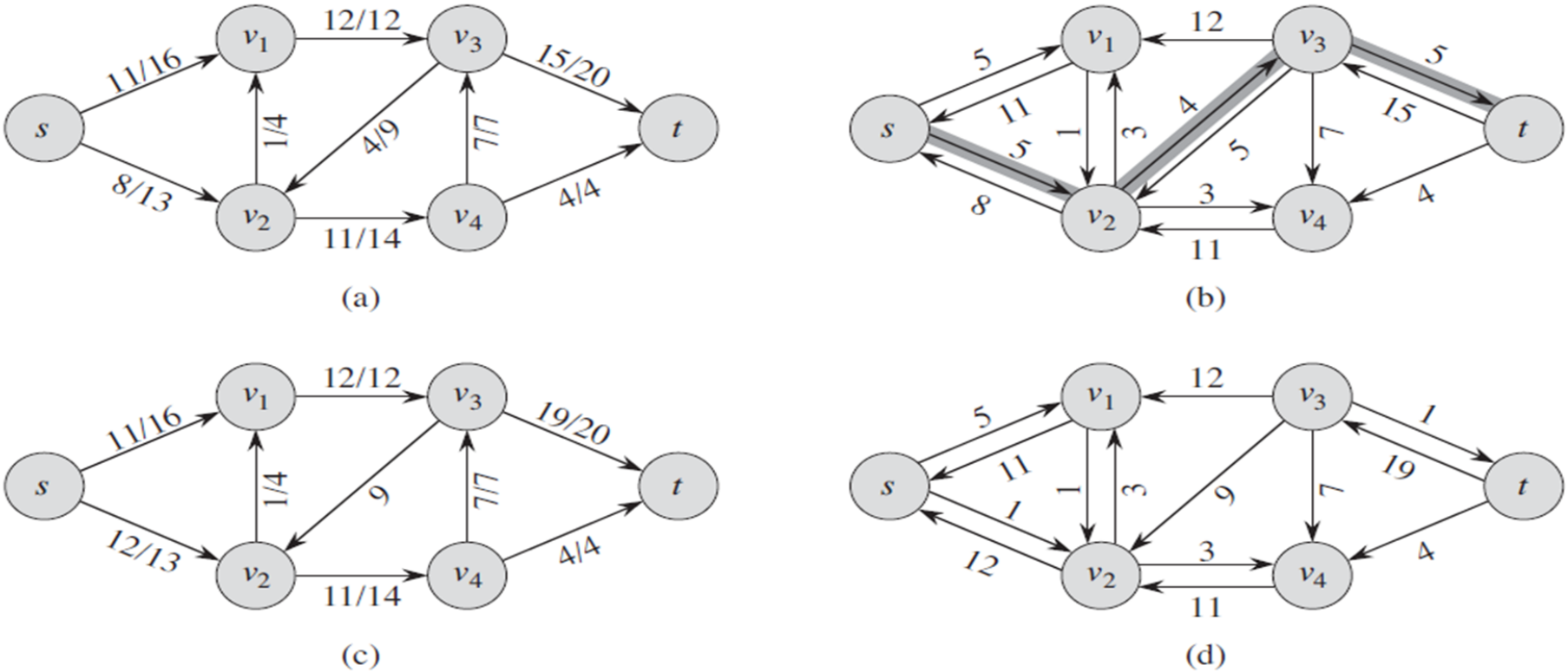


Figure 26.4 **(a)** The flow network G and flow f of Figure 26.1(b). **(b)** The residual network G_f with augmenting path p shaded; its residual capacity is $c_f(p) = c_f(v_2, v_3) = 4$. Edges with residual capacity equal to 0, such as (v_1, v_3), are not shown, a convention we follow in the remainder of this section. **(c)** The flow in G that results from augmenting along path p by its residual capacity 4. Edges carrying no flow, such as (v_3, v_2), are labeled only by their capacity, another convention we follow throughout. **(d)** The residual network induced by the flow in (c).

Augmenting Paths

- Given a flow network $G = (V, E)$ and a flow f, an **augmenting path** is a simple path from s to t in the residual network G_f.

- **Residual capacity** of p: the maximum amount of net flow that can be shipped along the edges of an augmenting path p.

$$c_f(p) = \min\{\, c_f(u, v) : (u, v) \text{ is on } p \,\}.$$



The residual capacity is 1.

The Ford–Fulkerson Method

- We call it a "method" rather than an "algorithm" because it encompasses several implementations with differing running times.
- The Ford–Fulkerson method depends on three important ideas: **residual networks**, **augmenting paths**, and **cuts.**
- **FORD–FULKERSON–METHOD**(G, s, t)
  1. Initialize flow f to 0.
  2. **while** there exists an **augmenting path** p in the residual network G_f
  3. Augment flow f along p.
  4. **return** f
- Here, G_f is the **residual network.**

The Ford–Fulkerson Method

- The Ford–Fulkerson method repeatedly augments the flow along augmenting paths until it has found a maximum flow.
- How do we know that when the algorithm terminates, we have actually found a maximum flow?
- The **max–flow min–cut theorem** tells us that a flow is maximum if and only if its residual network contains no augmenting path.
- To prove this theorem, we must first explore the notion of a **cut** of a flow network.

Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

Cuts of Flow Networks

- A **cut** (S, T) of a flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$.
- If f is a flow, then the **net flow** $f(S, T)$ across the cut (S, T) is defined as:

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

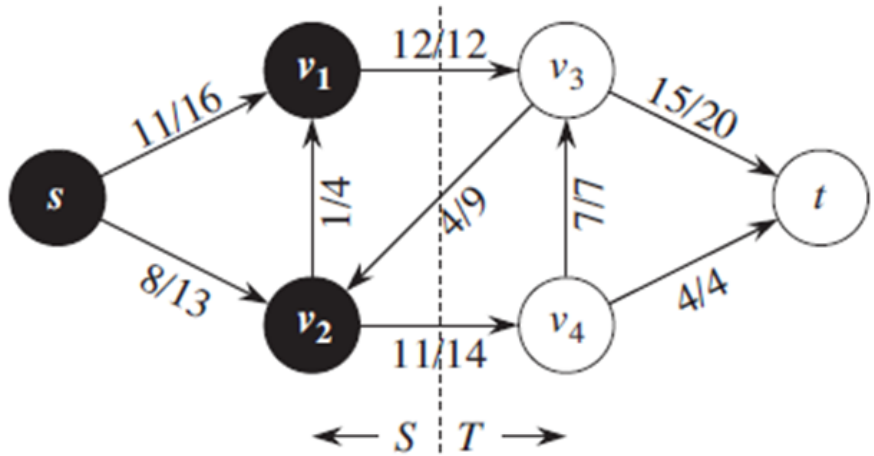
- The **capacity** of the cut (S, T) is:

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$$

- A **minimum cut** of a network is a cut whose capacity is minimum over all cuts of the network.

Example:

$$f(v_1, v_3) + f(v_2, v_4) - f(v_3, v_2) = 12 + 11 - 4 = 19$$
$$c(v_1, v_3) + c(v_2, v_4) = 12 + 14 = 26.$$



Lemma 26.4. Let f be a flow in a flow network G with source s and sink t, and let (S, T) be any cut of G. Then the net flow across (S, T) is $f(S, T) = |f|$.

Proof. We can rewrite the flow-conservation condition for any node $u \in V - \{s, t\}$ as:

$$\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0. \tag{26.11}$$

Taking the definition of $|f|$ from Equation (26.1) and adding the left-hand side of Equation (26.11), which equals 0, summed over all vertices in $S - \{s\}$, gives:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \left(\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) \right).$$

Expanding and regrouping terms yields:

$$\begin{aligned} |f| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \sum_{v \in V} f(u, v) - \sum_{u \in S - \{s\}} \sum_{v \in V} f(v, u) \\ &= \sum_{v \in V} \left(f(s, v) + \sum_{u \in S - \{s\}} f(u, v) \right) - \sum_{v \in V} \left(f(v, s) + \sum_{u \in S - \{s\}} f(v, u) \right) \\ &= \sum_{u \in S} \sum_{v \in V} f(u, v) - \sum_{u \in S} \sum_{v \in V} f(v, u). \end{aligned}$$

Because $V = S \cup T$ and $S \cap T = \varnothing$, we can split each summation over V into summations over S and T to obtain

$$|f| = \sum_{v \in S} \sum_{u \in S} f(u, v) + \sum_{v \in T} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in S} f(v, u) - \sum_{v \in T} \sum_{u \in S} f(v, u)$$

$$= \sum_{v \in T} \sum_{u \in S} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) + \left(\sum_{v \in S} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in S} f(v, u) \right)$$

The two summations within the parentheses are actually the same, since for all vertices $x, y \in V$, the term $f(x, y)$ appears once in each summation. Hence, these summations cancel, and we have

$$|f| = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

$$= f(S, T).$$

Corollary 26.5. The value of any flow f in a flow network G is bounded from above by the capacity of any cut of G.

Proof. Let (S, T) be any cut of G and let f be any flow. By Lemma 26.4 and the capacity constraint,

$$\begin{aligned}
|f| &= f(S, T) \\
&= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) \\
&\leq \sum_{u \in S} \sum_{v \in T} f(u, v) \\
&\leq \sum_{u \in S} \sum_{v \in T} c(u, v) \\
&= c(S, T).
\end{aligned}$$

Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

Max-flow Min-cut Theorem

Theorem 26.6 (Max-flow Min-cut Theorem)
If f is a flow in a flow network $G = (V, E)$ with source s and sink t, then the following conditions are equivalent:

1. f is a **maximum flow** in G.
2. The residual network G_f contains **no augmenting paths**.
3. $|f| = c(S, T)$ for some cut (S, T) of G.

Proof: Max-flow ⇒ No Augmented Path i.e., (1) ⇒ (2)

Lemma 26.2. Let $G = (V, E)$ be a flow network, let f be a flow in G, and let p be an augmenting path in G_f. Define a function $f_p : V \times V \to \mathbb{R}$ by

$$f_p(u, v) = \begin{cases} c_f(p), & \text{if } (u, v) \text{ is on } p, \\ 0, & \text{otherwise.} \end{cases}$$

Then f_p is a flow in G_f with value $|f_p| = c_f(p) > 0$.

Corollary 26.3. Let $G = (V, E)$ be a flow network, let f be a flow in G, and let p be an augmenting path in G_f. Let f_p be defined as above, and suppose we augment f by f_p. Then the function $f \uparrow f_p$ is a flow in G with value

$$|f \uparrow f_p| = |f| + |f_p| > |f|.$$

Proof: Max-flow ⇒ No Augmented Path i.e., (1) ⇒ (2)

Goal: Show that if f is a **maximum flow**, then the residual network G_f contains **no augmenting paths**.

Idea in Simple Terms:

- Suppose, for contradiction, that f is a **maximum flow**, but there *still exists* an augmenting path p in G_f.
- Using **Lemma 26.2**, we can define a small flow f_p along that path p whose value is $|f_p| = c_f(p) > 0$.
- By **Corollary 26.3**, if we **augment** f by this f_p, the new flow becomes:

$$|f \uparrow f_p| = |f| + |f_p| > |f|.$$

- This means we have found a flow with a **greater value** than the so-called "maximum flow" — a contradiction.

Therefore, no augmenting path can exist in G_f if f is truly a maximum flow.

Proof: No Aug. Path $\Rightarrow |f| = c(S, T)$ i.e., (2) $\Rightarrow$ (3)

Goal: Show that if the residual network G_f has **no augmenting paths**, then $|f| = c(S, T)$ for some cut (S, T) of G.

Key Idea:

- If there is **no path from** s **to** t in G_f, we define:

$$S = \{v \in V : \text{there exists a path from } s \text{ to } v \text{ in } G_f\}, \quad T = V - S.$$

  This (S, T) forms a valid **cut** (since $s \in S$ trivially, and $t \in T$).

- For any pair of vertices $u \in S$ and $v \in T$:
  - $f(u, v) = c(u, v)$
    (otherwise $(u, v) \in E_f$, which would place $v \in S$).
  - $f(v, u) = 0$
    (otherwise $c_f(u, v) > 0 \implies (u, v) \in E_f$, placing $v \in S$).

- Therefore, the total net flow across the cut is:

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) = \sum_{u \in S} \sum_{v \in T} c(u, v) - \sum_{v \in T} \sum_{u \in S} 0 = c(S, T).$$

Hence, by Lemma 26.4, $|f| = f(S, T) = c(S, T)$.

$|f| = c(S, T) \Rightarrow$ Maxflow i.e., (3) $\Rightarrow$ (1)

Goal: Show that if $|f| = c(S, T)$ for some cut (S, T), then f is a **maximum flow** in G.

Key Idea:

- From **Lemma 26.4:** For any cut (S, T), the net flow across the cut equals the flow value:
$$|f| = f(S, T).$$
- From **Corollary 26.5:** The value of any flow f is always **less than or equal to** the capacity of every cut:
$$|f| \leq c(S, T).$$
- If for some particular cut (S, T), we find that
$$|f| = c(S, T),$$
then f achieves the **maximum possible value** — no other flow can exceed this capacity.

Therefore, f is a **maximum flow** in G.

The Basic Idea of Ford–Fulkerson Algorithm

Goal: Compute the **maximum flow** in a flow network $G = (V, E)$.

Key Concept:

- The algorithm starts with an initial flow $f(u, v) = 0$ for all edges.
- It repeatedly finds a path p from the **source (s)** to the **sink (t)** in the **residual network** G_f.
- The path p is called an **augmenting path**.

How it works:

1. Find a path p from s to t in the residual graph G_f.
2. Determine the **bottleneck capacity:**

$$c_f(p) = \min\{c_f(u, v) : (u, v) \text{ is on } p\}.$$

3. Increase the flow along this path by $c_f(p)$.
4. Update both forward and backward edges to reflect the new flow.

Repeat until no augmenting path exists in G_f.

Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

Ford–Fulkerson Algorithm (Pseudocode)

Ford–Fulkerson(G, s, t)

1. For each edge $(u, v) \in G.E$: set $(u, v).f = 0$
2. **While** there exists a path p from s to t in residual network G_f:
   1. Compute residual capacity along p:

$$c_f(p) = \min\{c_f(u, v) : (u, v) \in p\}$$

   2. For each edge (u, v) in p:
      - If $(u, v) \in E$, then $(u, v).f = (u, v).f + c_f(p)$
      - Else $(v, u).f = (v, u).f - c_f(p)$
3. **Return** f

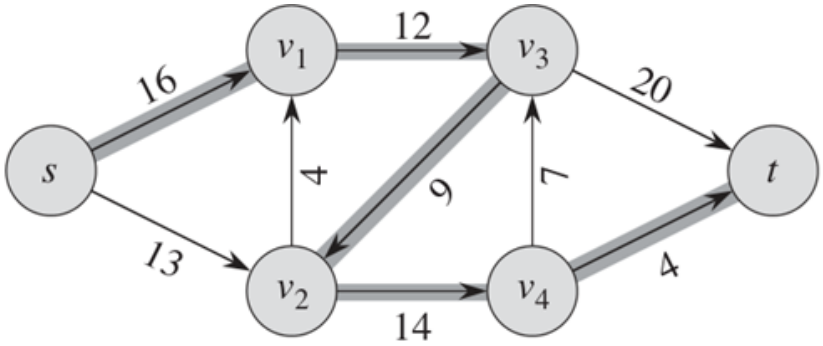
- The algorithm "pushes" flow along paths with available capacity.
- If an edge becomes full, we can "undo" part of its flow using the reverse edge.
- Process continues until no more s–t paths exist in G_f.

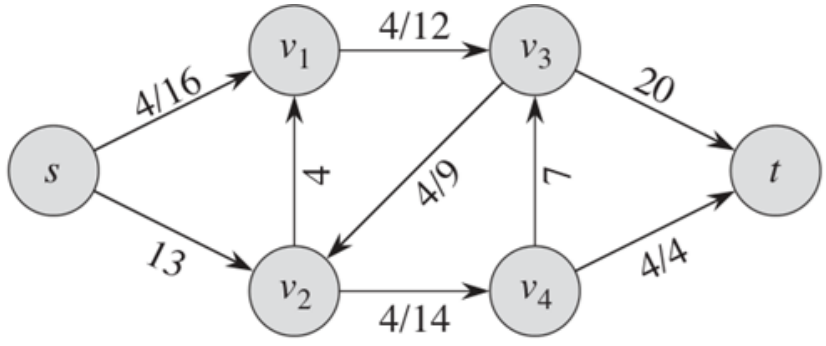
Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

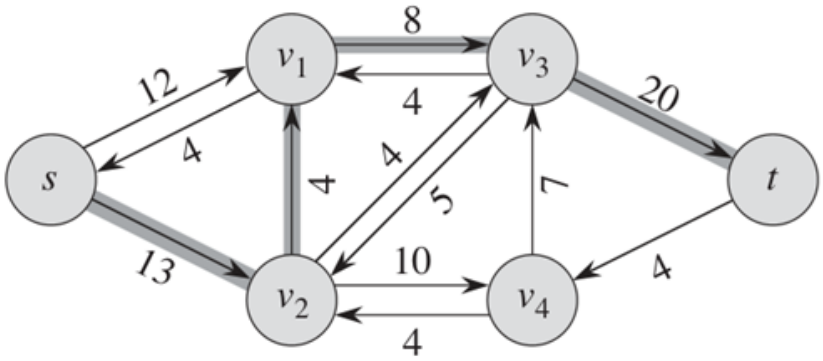
Example

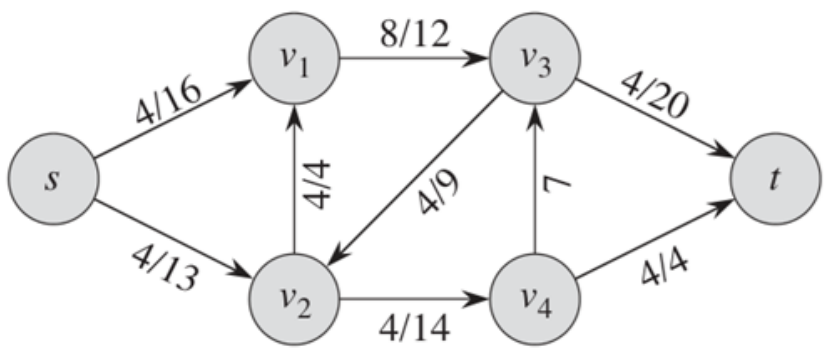
(a)


s
v_1
v_2
v_3
v_4
t
16
13
12
4
9
14
7
20
4

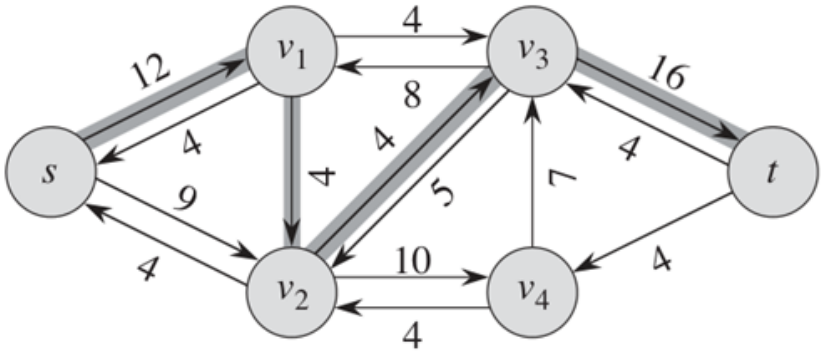

s
v_1
v_2
v_3
v_4
t
4/16
13
4/12
4
4/9
4/14
7
20
4/4

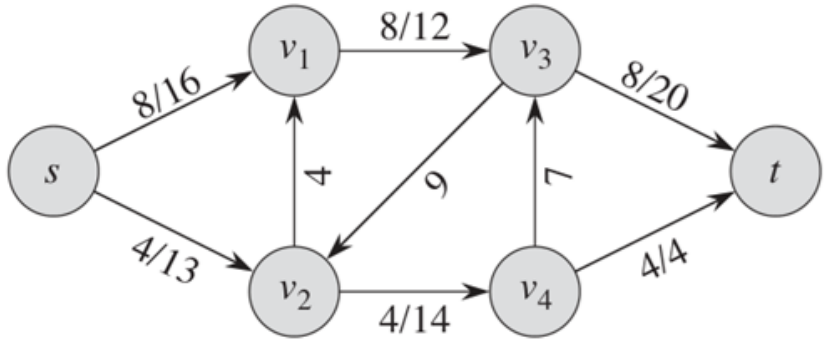
(b)


s
v_1
v_2
v_3
v_4
t
12
4
13
8
4
4
4
5
10
4
7
20
4


s
v_1
v_2
v_3
v_4
t
4/16
4/13
8/12
4/4
4/9
4/14
7
4/20
4/4

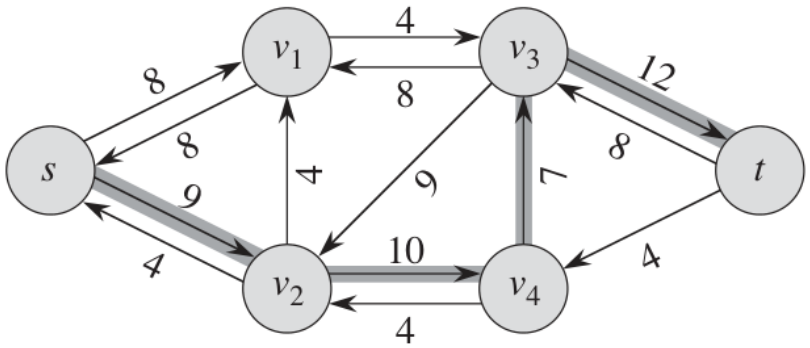
(c)

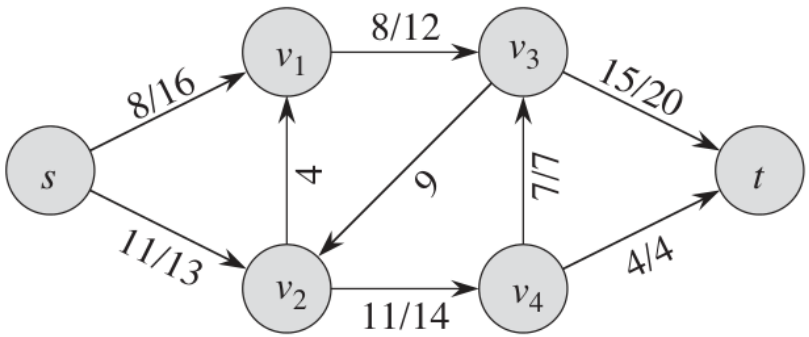

s
v_1
v_2
v_3
v_4
t
12
4
9
4
4
8
4
4
5
10
4
7
16
4
4


s
v_1
v_2
v_3
v_4
t
8/16
4/13
8/12
4
9
4/14
7
8/20
4/4

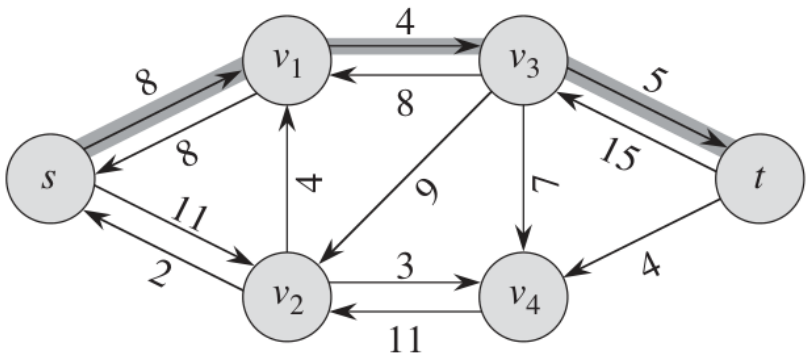
Example (Cont'd)

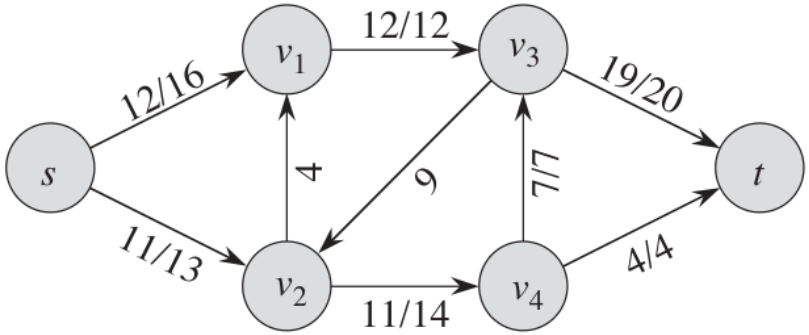
(d)



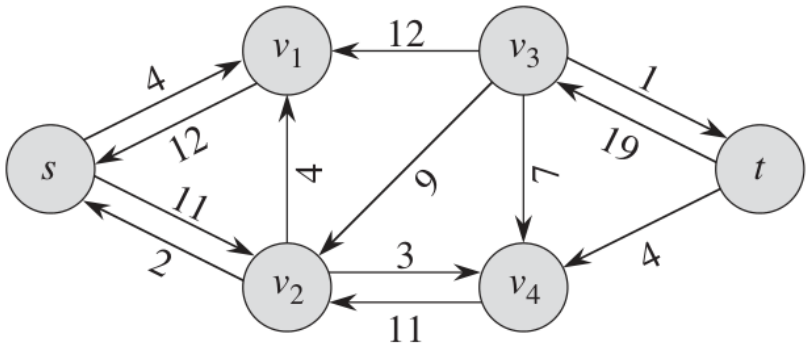


(e)

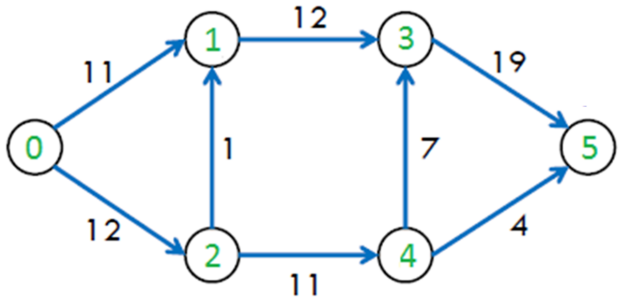


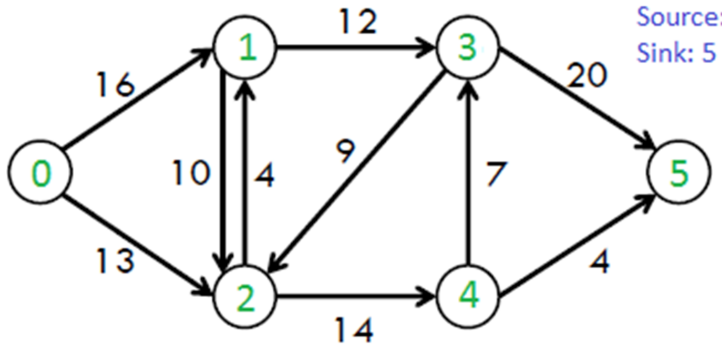


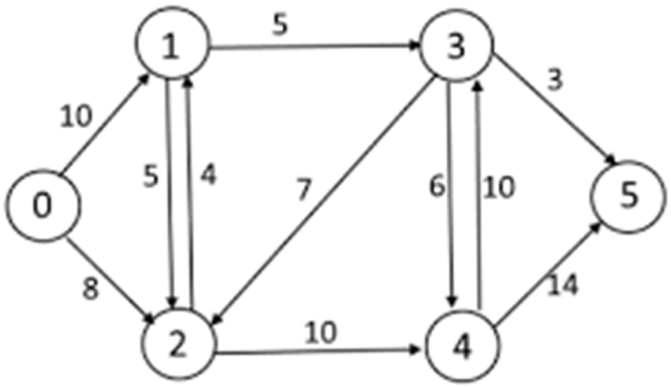
(f)



Try These







Source : 0
Sink : 5

Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

Bipartite Matching: Application of Flow Network

Definition 1.7.1: Bipartite Graph
A **bipartite graph** $G = (V, E)$ is a graph in which the vertex set V can be divided into two disjoint subsets X and Y such that every edge $e \in E$ has one endpoint in X and the other endpoint in Y.

Example: A bipartite graph with 5 left nodes and 4 right nodes.

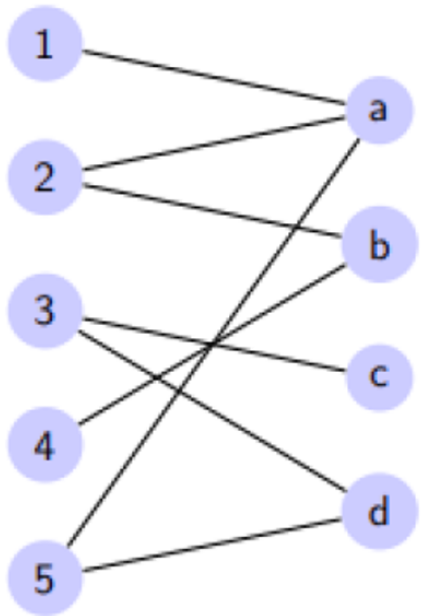


Figure 1: Bipartite Graph Example

Matching in Bipartite Graph

Definition 1.7.2: Matching
A **matching** M is a subset of edges E such that each vertex in V appears in at most one edge in M.

Definition 1.7.3: Maximal Matching
A **maximal bipartite matching** is the largest subset of edges such that no two selected edges share a common vertex.

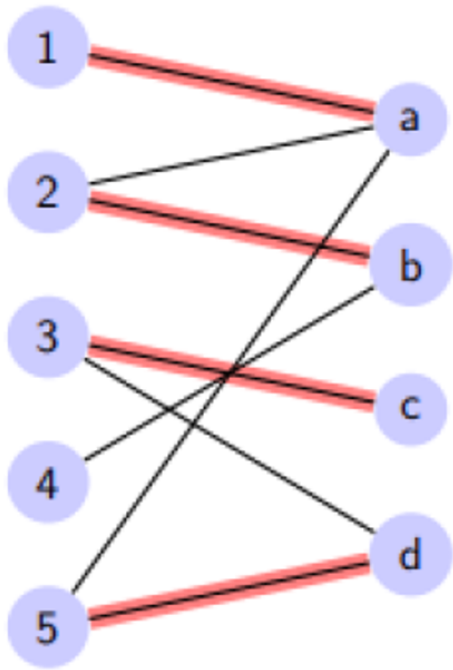


Figure 2: Matching Bipartite Graph

Reducing Bipartite Matching to Flow Network

Steps to Convert Bipartite Matching into Flow Problem:

1. Given bipartite graph $G = (A \cup B, E)$, direct edges from A to B.
2. Add two new vertices: source s and sink t.
3. Add an edge from s to every vertex in A.
4. Add an edge from every vertex in B to t.
5. Assign capacity 1 to all edges.
6. Solve the maximum flow problem on the resulting network G'.

Example: Converting Bipartite Matching to Flow Network

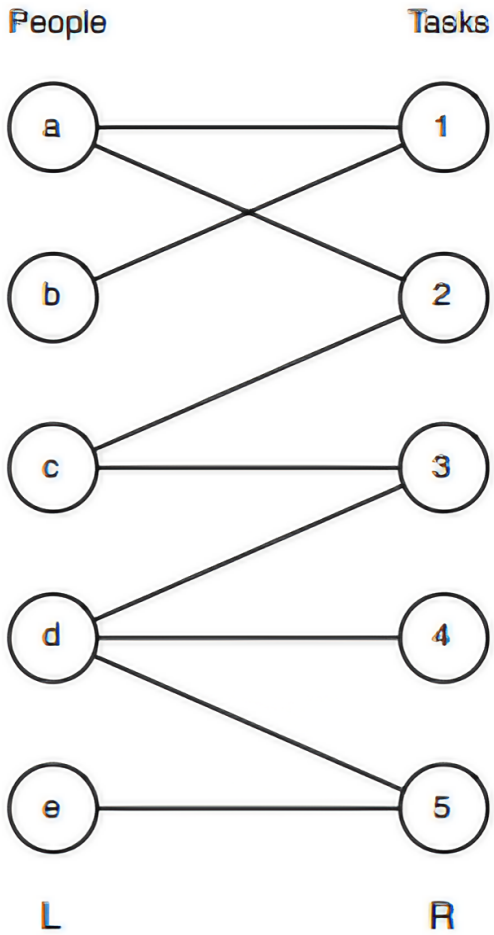


Figure 3: Undirected Bipartite Graph

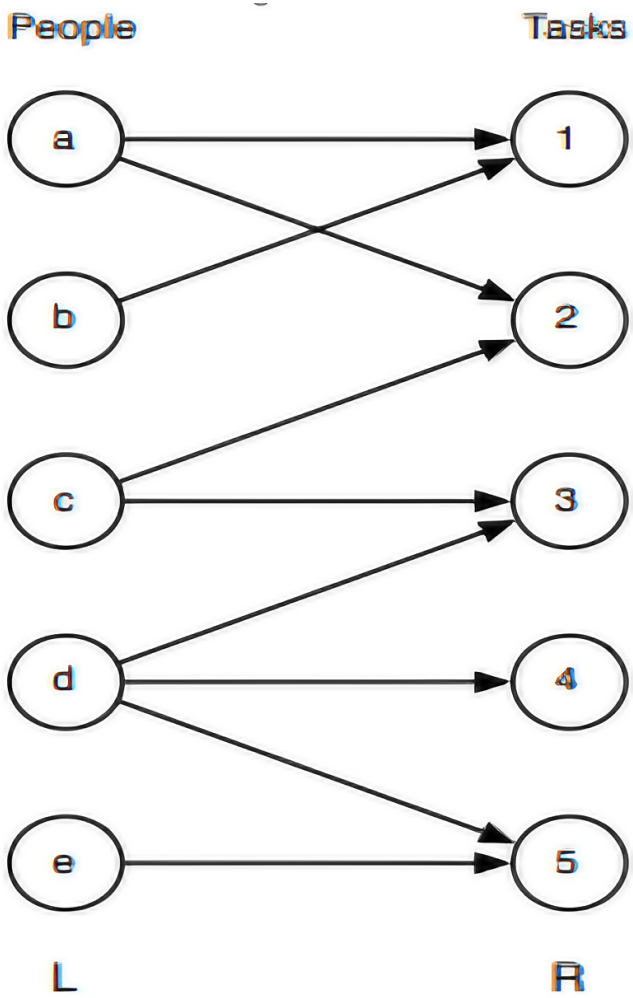


Figure 4: Directed Bipartite Graph

Adding Source and Sink to Build Flow Network

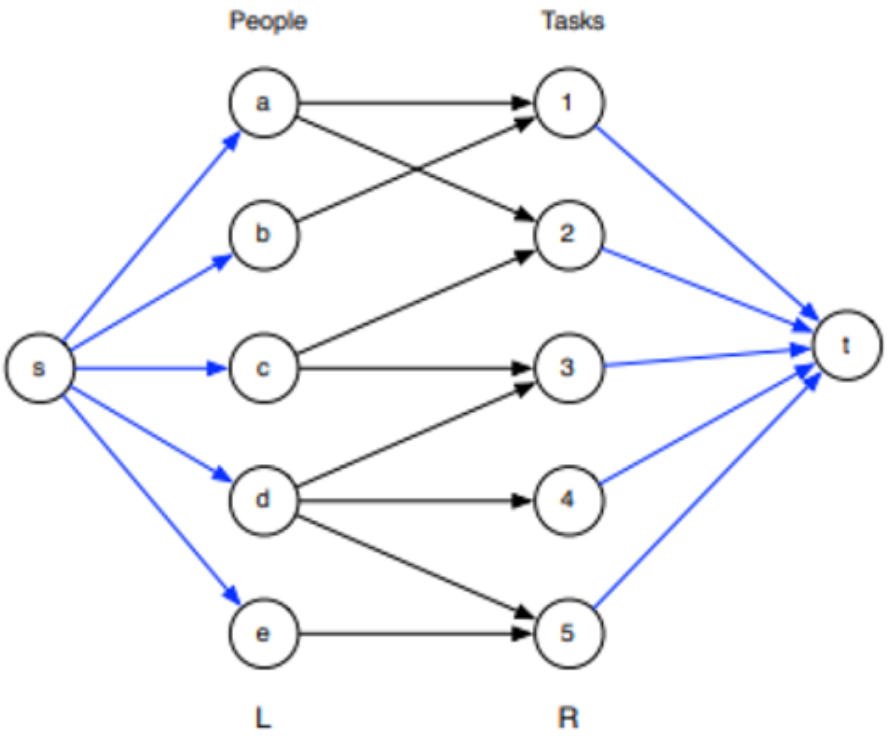


Figure 5: Reduced Flow Network

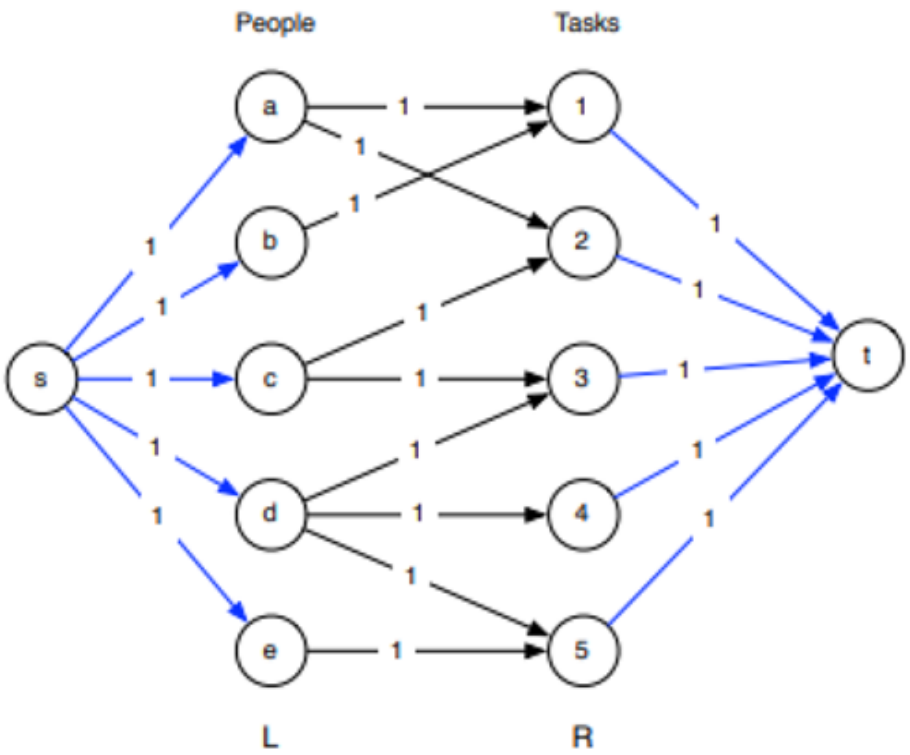


Figure 6: Flow Network with Capacity = 1

We can now apply the **Ford–Fulkerson Method** to find the **maximum matching** in this flow network.

Table of Contents

1. Introduction
2. Flow networks
3. The Ford–Fulkerson Method
   - Residual Networks
   - Augmenting Paths
   - Ford–Fulkerson Method
4. Cuts of Flow Networks
5. Max-flow Min-cut Theorem
6. Ford–Fulkerson Algorithm Final Version
7. Example and Exercise
8. Bipartite Matching: Application of Flow Network
9. More Questions

Practice Exercises

1. Consider a flow network with the given edge capacities: $c(s,2) = 2, c(s,3) = 13, c(2,5) = 12, c(2,4) = 10, c(3,4) = 5, c(3,7) = 6, c(4,5) = 1, c(4,6) = 1, c(6,5) = 2, c(6,7) = 3, c(5,t) = 6, c(7,t) = 2$.
   - Draw the flow network.
   - Run Ford–Fulkerson and show all residual graphs.
   - Show the minimum cut.
2. What is the maximum s–t flow? Draw the final residual graph (Figure next slide).
3. Show the minimum cut for the flow-network (Figure 1.13).
4. Explain why maximum matching = maximum flow in bipartite graphs.

Practice Exercises

